

Functional Complexity Quantification for Transdisciplinary Engineered Systems

Lal BAHADUR¹ and Eric REBENTISCH

Massachusetts Institute of Technology, 77, Massachusetts Ave, MIT Building E40,
Cambridge, MA 02139, USA

Abstract. Accurately assessing the scope of effort required to complete the development of new transdisciplinary engineered systems is important to creating a development strategy and planning the allocation of resources over the development lifecycle. Doing this in the earliest phases of the development lifecycle is particularly challenging, where more may be known about the problem space than the solution space. The desired system functions may be characterized, but the elements of the system or even the system's physical architecture may not yet have been established. One potential way to characterize the scope of work and to differentiate between different system concepts is to characterize their respective functional complexity. Functional complexity has been explored from several different perspectives, unfortunately, often without producing practical methods to use it in a transdisciplinary engineering context. This research identifies gaps in existing functional complexity assessment methods through a review of academic literature and insights from a practitioner's perspective. A set of measures and a framework for characterizing the relative complexity of a system is presented, along with possible scenarios where this approach might be used during the early stages of the development of a new transdisciplinary system when little more than its desired functions are known.

Keywords. Transdisciplinary Engineering, Functional Complexity, Functional Interdependency, Complex Systems Design, Complexity Framework, Engineered Complex System

Introduction

Developing new Transdisciplinary Engineered (TE) complex systems requires early planning and estimating efforts, capability, and resources. Such efforts in the early phases of the development of the lifecycle are more challenging, as we know less about the solution space and more about the problem space. Understanding the problem space may lead us to successfully characterize the system functions while system physical architectures or system elements are yet to be established or matured. The characterization of system functions may enable system engineers to quantify the functional complexity of engineered systems to identify cost, scope and estimate efforts and resources more precisely and accurately to develop strategies and more effectively allocate resources over the development lifecycle [1]. This paper presents and evaluates parameters that may be useful to quantify functional complexity to help engineers design complex systems in transdisciplinary domains. It discusses evaluation metrics for

¹ Corresponding Author, Email: lbpatel7@mit.edu.

functional complexity that should be a priority for future research to create a pragmatic means of calculating functional complexity of transdisciplinary systems.

Often, at the start of the system development efforts, system engineers have some visibility of the intended system functions and the supporting structures. To better assess resources, cost, and schedule, system complexity can be quantified based on parameters of the problem space. System complexity can be assessed from both structural and functional perspectives. However, during the initial stages of a system lifecycle, the systems engineers have more knowledge of intended system functions coming from stakeholder needs analysis since the system architecture may not yet be developed. The non-availability of system architecture makes structural complexity of little use during the initial stages of the system's lifecycle. While there has been substantial progress in developing methods to quantify structural complexity, functional complexity remains abstractly defined. The current limited methods to quantify functional complexity are inadequate for engineering practice.

The approach introduced and evaluated in this paper seeks to make functional complexity usable to engineers during the early stages of system development. Functional complexity concepts are reviewed, along with components of functional complexity. Practical methods and parameters to quantify the functional complexity of the system are proposed using a framework based on a Technical Requirements Document (TRD). This paper also discusses how system engineers can compare one system with another using functional complexity and reviews whether functional complexity correlates with project/ system scope, cost, and/ or performance. A comprehensive and streamlined method to quantify functional complexity early on may help organizations and transdisciplinary engineering teams make informed choices and optimize designs and structures of real-world complex engineered systems.

Quantifying functional complexity in early system development phases may also provide insights to transdisciplinary engineering teams to better manage risks, evolve systems for simplified in-service support and maintenance, perform tests and validations to reduce design failures, and optimize system performance. Thus, it may help to reduce total project costs, advance innovations in the system, and help in functional optimization through trade-offs of various structural matrices. The paper evaluates factors that make a system functionally complex.

Before looking into a mechanism for quantifying functional complexity, it is important to explore functional complexity as a construct and understand why it may be useful to transdisciplinary engineers.

One way to define functional complexity is by the number of functions it has. This could be deduced from a list of elements, their interactions, and emergent functions that make functional sets unique in different systems. Additional factors of evaluation would be the criticality of tasks in each system. Take, for instance, the propulsion system of aircraft. A measure of functional complexity may increase if the number of missions grew [2]. One would expect to see higher functional complexity in a propulsion system designed with functions such as cruise variations, altitude variations, Vertical Take-Off and Landing (VTOL), and subsonic and supersonic cruise versus a single mission aircraft (with only one requirement). Likewise, additional factors of the operating environment, the relative difficulty of the project compared with previous projects (operationalized as a Complexity Adjustment Factor or CAF) [3], and sequence of functions would influence functional complexity. The framework proposed in this paper seeks to help an engineer

to explain what makes something complex and what factors play a role in this phenomenon.

1. Literature Review

We reviewed research papers using search terms: Functional Complexity (FC), quantifying FC, formula for FC, characterization of FC, attributes of FC, what makes systems more functionally complex, and CAF FC. From the search results, we identified 23 papers that provided methods for estimating FC and were generally applicable across different product domains (e.g., not exclusively valuable for software development).

Transdisciplinary Engineering (TE) systems require integrating diverse stakeholder inputs for systems development. Within this paradigm, the Concept of Operations (CONOPS) is a critical tool facilitating communication among stakeholders, including program managers, users, operators, and regulators. The CONOPS enables collaboration and engineering decisions that align with broader system objectives and operational needs. Our focus on transdisciplinary engineering stems from bridging domain-specific silos, creating shared understanding, and improving decision-making processes in complex engineering environments early on. For system engineers, artifacts such as Technical Requirements Documents (TRD) derived from CONOPS serve as a key means of communication with and insights regarding stakeholders and serves as a crucial tool for communication among program managers, users, operators, and regulators. Assessing functional elements and the complexities arising from their quantity, interplay, and emergent system behaviour can provide key insights into how to manage them.

Different theories and approaches exist in the literature regarding the characterization of functional complexity for TE systems. Albrecht et al. characterized system functions as ‘function points’ [4] by summing all function types and an assigned complexity measure. This approach relies on software-centric information to assign the complexity value to each function. Their studies have not considered the impact of other critical elements like system interconnectivity and hierarchy. Because of this, its usefulness is limited to software systems.

McCabe characterized functional complexity through an approach called Cyclomatic Complexity [1]. This method used parameters like functional edges, nodes, and connected components. However, this process is mathematically linear, meaning it does not address increased complexity effects resulting from increased functional elements. A cyclomatic approach is valid for software systems to predict the maintainability or error-proneness of software codes [1]. So, while this approach can estimate functional complexity to a degree, it has significant limitations when applied to complex systems.

Although limited to biopolymers and artificial life systems only, Hazen et al. characterized system complexity in terms of system function [5]. They used functional information as a complexity measure by calculating logarithmic function, which shows evidence of distinct solutions with different degrees of function [5]. Their study did not address how this could be used on systems that are being developed or in early stages, like non-biological physical systems. The Sosa et al. approach was based on engineered mechatronic systems and quantified functional complexity as the number of elements in a system and the degree of interconnectedness between these elements. They use size, connectivity, and system modularity as parameters. Unfortunately, these elements are

often not yet identified during initial program stages [6]. Shannon's approach uses system entropy to reflect its level of disorder or unpredictability. A higher entropy in Shannon's framework means higher uncertainty and increased functional complexity [7]. This makes sense, as a higher entropy would lead to more elements, connections, surrounding interfaces, and possible emergent system behaviour. These attributes would in turn lead to increased functional complexity. Shannon's entropy approach is a step toward quantifying functional complexity, though it does not specify the actual relationships between elements and how they map to the functional complexity.

Julie et al. approached functional complexity using a functional basis for engineering designs and classified function sets into primary, secondary, and tertiary hierarchical relationships [8]. This classification is helpful for separating functional element-level complexities into different categories that can be mapped to system elements. Bashir and Thomson used a sum of all the number of functions multiplied by their levels in functional hierarchies as a factor to determine product complexity [9]. The hierarchical approach in Bashir's model can be used to investigate functional relations and their linkage to the system to calculate functional complexity.

Keating has defined a complexity metric as an outcome of variables like the number of modules and interfaces [2]. Their complexity measure uses the number of modules and interfaces between these modules. While we have some level of information about both modules and interfaces during early system development stages, this research helps us use these characteristics to build a pragmatic functional complexity characterization. Kremer et al. reviewed and filtered fourteen metrics to recommend three papers from Bashir [9], Roy [10] and Keating et al. [2]. However, much of their research has been descriptive in nature and did not propose any direct approaches for quantifying functional complexity [11]. Raudberget et al. used set-based evaluation and change propagation to assess systems and model their architecture [12]. While the approach in the paper is a significant contribution, it lacks a conceivable quantitative analysis to compare TE systems in different domains.

Sushil et al. proposed an approach called function point using Complexity Adjustment Factor (CAF) and Unadjusted Functional Point (UFP) [3]. A CAF in this approach uses 14 criteria relating to the system's complexity relative to prior efforts based on a scale of 0 – 5 [3]. While CAF was introduced for software development applications, almost all the factors are well-suited for cyber-physical systems and usually identifiable at early program development stages. Elaine suggested seven-point criteria for software systems complexity relevant to functional complexity metrics [13]. Elaine's criteria can also be applied to cyber-physical systems because the parameters are not exclusive to the software development context.

1.1. Summary of the Literature Review

The literature reviewed in this paper focused on parameters ranging from the number of functions to weighting and topology that may be used to determine functional complexity. **Table 1** shows the main attributes of functional complexity from the literature review from software and hardware domains. Many FC attributes are from the information from the software domain, although **Table 1** also includes FC characteristics from hardware domains.

Table 1. Common themes/ approaches on functional complexity in current literature.

References	Parameters	Domain	Remarks
[14],[1], [4], [15], [16], [9], [2], [17], [5], [18]	Number of functions	Information, Hardware, Software	Functions also termed as nodes, entities
[1], [7], [8], [19], [2], [13], [5], [20], [18]	Interdependencies between functions	Information, Hardware, Software	Interdependencies also termed as number of edges, relationships
[19], [8], [9], [13]	Level/ hierarchy of functions	Hardware, Software	Also classified as Primary, Secondary and Tertiary functions
[4], [18]	Weighting of individual functions	Hardware, Software	Also includes number and likelihood of states in the system (entropy)
[7], 6, [11], [10], [13], [18], [3], [12]	Topology of functions	Information, Hardware, Software	Chain of functions, Feedback Loops

The studies reviewed here provide limited approaches for quantifying functional complexity and lack a pragmatic approach (from the perspective of a transdisciplinary engineering team) to calculate FC. Many of the factors proposed lack an obvious data source to enable FC quantification in early program stages. Many of the papers have only described a theory of functional complexity. For example, Shannon entropy [7] is a theoretical approach and abstraction while cyclomatic complexity [1] is directly applicable only for software systems.

Most of the papers that have attempted to characterize elements of functional complexity rely on inputs that are not available during early program phases, neither are there any standard documents that capture all the required information useful for early-stage system engineers. The literature reviewed here provides a basic framework using several functions [9], their inter-dependencies, levels [9], and weighting [4]. This makes the current quantification frameworks of limited use to engineers in early system development stages.

2. Application for Transdisciplinary Engineered Systems

During the early contractual bidding stage of engineered systems, there is limited information available to system engineers that could be used to predict functional complexity. One information source that is available early in a program and that describes the system's functions is the Technical Requirements Document (TRD). As shown in the [Figure 1](#), in the case of a system developer receiving TRD at the bidding stage to an OEM customer makes the process structured. The TRD is used as a reference document to capture functional requirements, wherein all system functional requirements are captured with an assigned identification number that can be tracked, revised, or removed with traceability and agreement with the stakeholders. [Figure 1](#) shows the flow of information about the system at the beginning of its development, from bid submission to defining a system boundary and the role of the TRD in capturing system functions and requirements by using customer needs into system requirements. It is worth noting that

the requirements translate into system functions, and then we start envisioning the supporting structure to perform these functions.

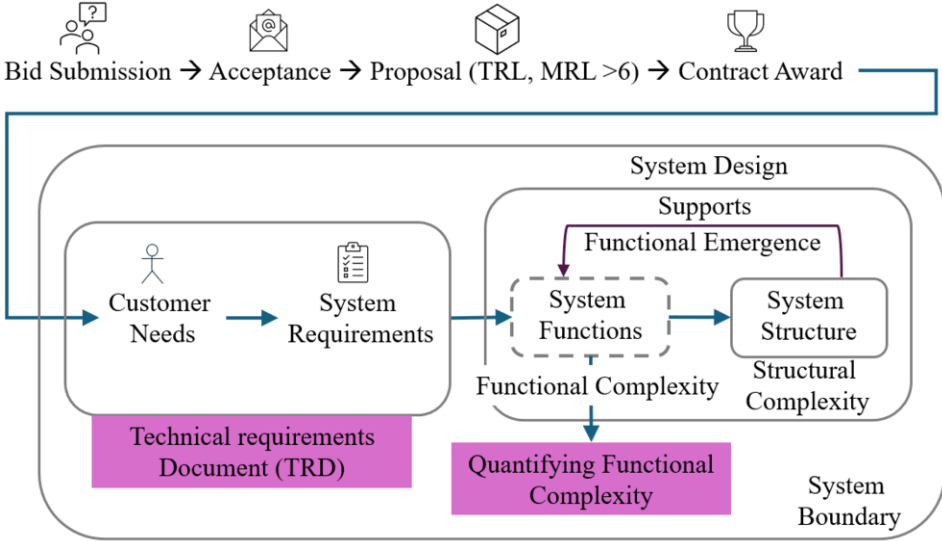


Figure 1. TRD framework for functional complexity quantification in early phase of system development stage

2.1. TRD Approach for Quantifying Functional Complexity of Engineered Systems

For the engineers during the early system development stage using the framework in **Figure 1**, the initial information is proposed to be captured and made available through a Technical Requirements Document (TRD) with all requirement identifications. The identification helps engineers track, revise, or remove requirements based on system development and stakeholder feedback. The next step after capturing all requirements is to filter all functional requirements to estimate functional complexity. The literature review identified several attributes/ characteristics of functional complexity. Extracting these characteristic elements from the TRD is proposed to estimate functional complexity.

To assess if the TRD have the information needed to calculate the functional complexity based on the reviewed literature, **Table 2** compares the factors from **Table 1** with information found in the TRD.

Table 2. Common themes/ approaches on functional complexity in TRD and comparison against literature.

Reviewed Literature			TRD	
All Characteristics	Domain	Parameter	Domain	Parameter
Number of functions	All domains	Yes	All domains	Yes
Non-linear interactions, delays	No	No	All domains	Yes
Frequency of function	No	No	All domains	Yes
Interdependencies between functions	All domains	Yes	All domains	Yes
Interconnection factor	No	No	All domains	Yes

Waiting fail-safe (murphy proofing) function	No	No	All domains	Yes
Level/ hierarchy of functions	Limited domains	Yes	All domains	Yes
Weighting of individual functions	Limited domains	Yes	Not available	Not available
Topology of functions	All domains	Yes	Not available	Not available
Type of linkage	No	No	All domains	Yes
Type and quantity of feedback loops	No	No	All domains	Yes

Table 2 shows that the TRD generally contains most of the information the literature suggests is required to assess functional complexity. Irrespective of whether the TRD is the most suitable information source, it is generally available early during system development.

2.2. Using TRD to Build a Model for Calculating Functional Complexity

The TRD represents all functional information in a controlled document, including the system's functional requirements, which informs the approach developed in the solution space. The potential solution options considered help to understand the functional complexity. The more constrained a function is, the more we must understand the solution space and use information from the solution space to characterize functional complexity.

To identify the nature of potential solutions, we review the extent to which solutions exist and assess the maturity and readiness of the available solutions through a technology readiness level (TRL) assessment [21]. Then, we investigate how constrained the potential solutions are. We can then define the attributes we must address to meet the desired system functions.

3. Framework for Calculating Functional Complexity of TE Complex Systems

After we have captured and evaluated all the functional requirements in the TRD, we start looking into ways to assess functional complexity and its implications for the resources and schedule needed for successful system development. Function complexity results from many variables that interplay to create dependencies and an emergent set of functions in complex systems. A pragmatic framework for quantifying functional complexity must draw from available information at this stage to practically help engineers quantify functional complexity. Based on the literature reviewed in the broader area of functional complexity and what can be extracted from documents such as the TRD, we compiled the main theme/ characteristics of functional complexity.

3.1. Characteristics of Functional Complexity

On the one hand the literature review provides a set of characteristics used for functional complexity while on the other hand we have TRD which is a source of early system

information. If we look at these two side by side, the TRD provides some, but not all the information required. Hence, we see the differences in characteristics that are usable during early stages based upon pragmatic considerations. To measure functional complexity, based upon the information available early on, and the source TRD, we look for the characteristics that are available and useful for quantification. The outcome of the literature review and characteristics helps to identify key characteristics for measuring functional complexity. As a result, the synthesized list below provides a set of those characteristics that are both useful and available early on for the engineered system:

- Number of functions, their level or hierarchy of the functions
- Functions that are in conflict, synchronized to each other, or neutral [6]
 - One function impacting other function- emergent behavior framework) [22]
- Non-linear Interactions that can have delays or create unpredicted interactions
 - Delays among functions
 - Unpredictable interaction of functions
- Weighting of individual functions
- The level or hierarchy modifies how one function may be more important than others. Not every function is weighed equally by the users. Often, one function is more critical than others in the eyes of the user. A classification of the function into three categories such as primary, secondary and tertiary helps to define the weighting of various functions. The classification might consider factors such as:
 - Frequency of function being executed
 - Number and likelihood of states in the system (entropy)
 - Complexity Adjustment Factor (CAF)/ Interconnection (I) Factor
 - Waiting for fail-safe (murphy proofing)
- Topology of functional elements considers attributes like
 - A CAF for interdependence of the topology, where some topologies are denser or entangled
 - Chain of functions, running several connections deep. The nature of linking within network
 - Number of Feedback Loops

The theory behind introducing a weighting factor comes from the fact that one function may have a higher weighting than others due to factors like the level of decomposition needed on specific functions, increasing complexity or the range of operating behavior changing functions – resulting in challenging/ complex systems. An example is a control valve or bolt that interfaces with an actuator, and a translating sleeve in aircraft engine thrust reversers may have a higher weighting factor. Finally, a factor named as the Interconnection (I) Factor is a solution-neutral factor capturing additional function linkages [17].

4. Conclusion

This paper reviewed several approaches to calculating functional complexity and summarized the relevant attributes. Quantifying functional complexity at the early stage of system development was viewed through the lens of the TRD, which provides timely

help to system engineers and acts as a tool to document engineering requirements, control changes in functional requirements, and extract functional information from the TRD. System engineers may use the framework in this paper to quantify functional complexity in a way that will help to estimate cost, schedule, and scope early in system development.

The pragmatic approach using TRD as a tool during early stages is novel [12]. The approach used here is not specific to one industry or system domain. It satisfies all key criteria from current literature, including Weyuker's criteria [13], and provides a tractable means for system engineers to compute functional complexity.

A limitation of the current paper in the spirit of a transdisciplinary approach with emergent complexities is that the current research is focused on systems engineers. If we include other stakeholders' feedback, then it may be transdisciplinary research in the true sense. This research is a work in progress towards operationalizing a formula to calculate Functional Complexity.

A call for action centers on future work toward developing a formula for functional complexity in a consistent method starting from the early stages for systems engineers and project managers. Future research may provide direction for a better estimation of functional complexity, building upon the characteristics in this paper and the literature.

References

- [1] T. J. McCabe, A complexity measure, *IEEE Transactions on software Engineering*, no. 4, pp. 308–320, 1976.
- [2] M. Keating, Measuring design quality by measuring design complexity, in *Proceedings IEEE 2000 First International Symposium on Quality Electronic Design (Cat. No. PR00525)*, Mar. 2000, pp. 103–108. doi: 10.1109/ISQED.2000.838861.
- [3] S. Kumar, R. Rastogi, and R. Nag, Limitations of Function Point Analysis in Multimedia Software/Application Estimation, in M. N. Hoda, N. Chauhan, S. M. K. Quadri, and P. R. Srivastava, (Eds.), *Software Engineering*, Springer, Singapore, 2019, pp. 383–392. doi: 10.1007/978-981-10-8848-3_36.
- [4] A. J. Albrecht and J. E. Gaffney, Software Function, Source Lines of Code, and Development Effort Prediction: A Software Science Validation, *IEEE Transactions on Software Engineering*, vol. 9, no. 6, pp. 639–648, Nov. 1983, doi: 10.1109/TSE.1983.235271.
- [5] R. M. Hazen, P. L. Griffin, J. M. Carothers, and J. W. Szostak, Functional information and the emergence of biocomplexity, *Proceedings of the National Academy of Sciences*, vol. 104, no. suppl_1, pp. 8574–8581, May 2007, doi: 10.1073/pnas.0701744104.
- [6] M. E. Sosa, S. D. Eppinger, and C. M. Rowles, The Misalignment of Product Architecture and Organizational Structure in Complex Product Development, *Management Science*, vol. 50, no. 12, pp. 1674–1689, 2004.
- [7] C. E. Shannon, A mathematical theory of communication, *The Bell System Technical Journal*, vol. 27, no. 4, pp. 623–656, Oct. 1948, doi: 10.1002/j.1538-7305.1948.tb00917.x.
- [8] J. Hirtz, R. B. Stone, D. A. McAdams, S. Szykman, and K. L. Wood, A functional basis for engineering design: Reconciling and evolving previous efforts, *Res Eng Design*, vol. 13, no. 2, pp. 65–82, Mar. 2002, doi: 10.1007/s00163-001-0008-3.
- [9] H. A. Bashir and V. Thomson, Estimating design complexity, *Journal of Engineering Design*, vol. 10, no. 3, pp. 247–257, 1999.
- [10] R. Roy, R. Evans, M. J. Low, and D. K. Williams, Addressing the impact of high levels of product variety on complexity in design and manufacture, *Proceedings of the Institution of Mechanical Engineers. Part B, Journal of engineering manufacture*, vol. 225, no. 10, pp. 1939–1950, 2011, doi: 10.1177/0954405411407670.
- [11] J. Crespo-Varela, G. Kremer, C. Tucker, and L. Medina, An Analysis of Complexity Measures in Product Design and Development, presented at the *Proceedings of the ASME Design Engineering Technical Conference*, Aug. 2012. doi: 10.1115/DETC2012-71309.
- [12] D. Raudberget, C. Levandowski, O. Isaksson, T. Kipouros, H. Johannesson, and J. Clarkson, Modelling and assessing platform architectures in pre-embodiment phases through set-based

- evaluation and change propagation, *AOP*, vol. 3, no. 3,4, pp. 203–221, Dec. 2015, doi: 10.3233/AOP-150052.
- [13] E. J. Weyuker, Evaluating Software Complexity Measures, *IEEE Transactions on Software Engineering*, vol. 14, no. 9, pp. 1357–1365, Sep. 1988, doi: 10.1109/32.6178.
 - [14] M. Dzaferagic, N. Kaminski, N. McBride, I. Macaluso, and N. Marchetti, A functional complexity framework for the analysis of telecommunication networks, *Journal of Complex Networks*, vol. 6, no. 6, pp. 971–988, Dec. 2018, doi: 10.1093/comnet/cny007.
 - [15] M. Li and P. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*. in *Texts in Computer Science*. Springer, New York, 2008. doi: 10.1007/978-0-387-49820-1.
 - [16] M. U. Thomas, Review of Elements of Applied Stochastic Processes, *Interfaces*, vol. 34, no. 4, pp. 321–322, 2004.
 - [17] H. W. Jones, The System Complexity Metric (SCM) Predicts System Costs and Failure Rates, [https://ntrs.nasa.gov/api/citations/20205005154/downloads/%20223%20A%20System%20Complexity%20Metric%20\(SCM\)%20rev%201.pdf](https://ntrs.nasa.gov/api/citations/20205005154/downloads/%20223%20A%20System%20Complexity%20Metric%20(SCM)%20rev%201.pdf), accessed July, 10 2025.
 - [18] W. ElMaraghy, H. ElMaraghy, T. Tomiyama, and L. Monostori, Complexity in engineering design and manufacturing, *CIRP Annals*, vol. 61, no. 2, pp. 793–814, 2012.
 - [19] M. E. Sosa, S. D. Eppinger, and C. M. Rowles, A Network Approach to Define Modularity of Components in Complex Products, *Journal of Mechanical Design*, vol. 129, no. 11, pp. 1118–1129, Jan. 2007, doi: 10.1115/1.2771182.
 - [20] L. Lavazza, A. Locoro, and R. Meli, Software Development and Maintenance Effort Estimation Using Function Points and Simpler Functional Measures, *Software*, vol. 3, no. 4, Art. no. 4, Dec. 2024, doi: 10.3390/software3040022.
 - [21] *TRL and MRL Definitions - Managing Technology and Product Development Programmes* - Wiley Online Library. Accessed: May 29, 2025. [Online]. Available: <https://onlinelibrary-wiley-com.libproxy.mit.edu/doi/10.1002/9781119517283.app1>
 - [22] A. Salado and R. Nilchiani, A Research on Measuring and Reducing Problem Complexity to Increase System Affordability: From Theory to Practice, *Procedia Computer Science*, vol. 44, pp. 21–30, Jan. 2015, doi: 10.1016/j.procs.2015.03.037.
 - [23] J. Sterman, *Business dynamics : systems thinking and modeling for a complex world*. Irwin/McGraw-Hill, Boston, 2000.